Architecture Justification – Academic Integrity, Regrade Appeals & Cohort Analytics

Name : Mohammed Sinan M T **SRN :** PES1UG24AM165
Section : C

Architecture Selection: We chose Layered Architecture for the Academic Integrity, Regrade Appeals & Cohort Analytics System (Automated Rubric Assignment Evaluator).

Architectural Choice: Layered Architecture with three layers. Presentation: Web Portal UI. Business: Access Control & API Gateway, Similarity Scan Service, Integrity Case Manager, Regrade & Score Revision Manager, Analytics & Export, and Notification. Data: Database Component. The Presentation Layer reaches the Business Layer only through the gateway (ISecureAPI), and only the Business Layer can reach the database. Components talk through interfaces such as IScanJob, IScanReport, IReportQuery and IAnalyticsRefresh.

Reason 1 – One consistent record for due process: FR-002 to FR-004 require every case transition, actor and timestamp to be logged and every superseded score to be retained, and FR-005 requires exports to match the dashboard row for row. With one Data Layer, the case, appeal and score-history components write to the same transactional store, so history and audit log cannot drift apart. Microservices would give each service its own data and add the distributed-consistency problem the handout lists as a con.

Reason 2 – Right size for the scenario: The system has two roles (Student, Faculty Evaluator), seven requirements and a cohort of 500 submissions, so independent deployment and scaling would add operational cost without benefit. A plain Client-Server design would be simple but would give no internal separation of scan, case and appeal logic. Layers keep each component replaceable through its interface, for example a different similarity algorithm behind IScanReport.

Security Advantage: The Presentation Layer has no path to business services or data except through the Access Control & API Gateway (ISecureAPI, HTTPS with TLS 1.2+). The gateway authenticates every call, checks the role, and filters by ownership, so a student only receives their own similarity report and case, and identified exports need an elevated-role action (NFR-001). Because there is one choke point, the NFR-001 access test (a student account cannot retrieve another student's report, case or ranked position) is run against a single component.

Performance Benefit: The Similarity Scan Service sits behind IScanJob and runs as an asynchronous background job. The quadratic comparison of 500 submissions against the current cohort and two previous offerings (NFR-002, 15 minutes) therefore never blocks the interactive evaluator queue. Analytics reads through its own IReportQuery interface and is refreshed through IAnalyticsRefresh after each score revision, which keeps dashboard queries within the 5-second budget.

Trade-off: Layering adds an extra hop and couples the layers more tightly, and individual components scale less freely than microservices. At this scale that is acceptable, and the Scan Service can later move to a separate worker because it is already isolated behind an interface.